

Trustworthy Machine Learning, 2026

Supervisors: Adam Dziedzic and Franziska Boenisch

Teaching Assistants: Maitri Shah and Nima Dindarsafa



Adversarial ML: Robustness

Task: Train an image classifier that is robust against adversarial attacks

Machine learning models are vulnerable to adversarial examples, which are inputs perturbed in subtle and often imperceptible ways that cause the model to make incorrect predictions. Such vulnerabilities pose serious risks in safety-critical applications such as medical diagnosis, autonomous driving, and security systems. In this task, your goal is to train a classifier that maintains high accuracy on both clean and adversarially perturbed inputs.

Task explanation video can be found [here](#).

What you get:

- A training dataset of 50,000 labeled images of shape 3x32x32 across 9 classes, provided as a .npz file [Link](#)

What you don't have access to:

- The private test set used for final evaluation
- The attack parameters and implementation used on the server side

What's your task:

Train an image classifier using one of the allowed model architectures that achieves the highest possible unified score. Your model must achieve a clean accuracy greater than 50% to be accepted for evaluation. Submissions below this threshold will be rejected.

Your model will be evaluated against adversarial examples generated using standard adversarial attack methods. The goal is to build a model that is genuinely robust, not one that overfits to a specific attack.

Allowed model architectures (from torchvision.models):

- resnet18
- resnet34
- resnet50

Your model must output logits for 9 classes and accept inputs of shape 3x32x32.

Evaluation Metric

Submissions are evaluated using:

$$Score = 0.5 * clean\ accuracy + 0.5 * robustness\ score$$

Where:

- Clean accuracy: fraction of unperturbed test samples correctly classified
- Robustness accuracy: fraction of adversarially perturbed test samples correctly classified

Higher values indicate better performance. Note that there exists a fundamental tradeoff between clean and robust accuracy. Both are equally weighted in the final score.

Dataset Details

You are provided with a training set of 50,000 images of shape 3x32x32 across 9 classes. The images are stored as uint8 values in a .npz file. Labels are encoded as integers in the range 0 to 8. The evaluation set comes from the same distribution as the training set. No other information about the dataset is provided. (see task_template.py for loading example)

Your Starting Point

To start you off, we provide a [task_template.py](#) that shows you how to load the dataset, define your model, and save your state dict. We also provide a [submission.py](#) that handles submitting your model to the evaluation server.

All data files can be found at [HuggingFace](#).

How to submit your results?

- We are provided a unique API-key to every participating team. You are supposed to only use this token to submit your results to our server.
- Your submission must be a .pt file containing your model's state dict, along with the model architecture name in the model-name field. You are supposed to submit your model using the code provided in submission.py.
- Remember to replace [YOUR_API_KEY_HERE](#) with your actual API token (keep the double quotes), replace [PATH/TO/YOUR/MODEL.pt](#) with the path to your saved model, and set [MODEL_NAME](#) to the architecture you used (resnet18, resnet34, or resnet50). Also set SUBMIT to True if not already.
- To avoid the possibility of brute-forcing, only one submission can be made every 60 minutes. If the submission was unsuccessful due to an error, this cooldown period will only be 2 minutes long.

Submission Format

Submit a .pt file containing only the model state dict, not the full model instance. The model must be one of the allowed architectures with the final layer replaced to output 9 classes. We strongly recommend running the assertions in task_template.py locally before submitting. To submit, set your API token and model path in submission.py and run it .

Common reasons for rejection:

- Clean accuracy below 50%
- Wrong output shape (must be 9 classes)
- Wrong input shape (must be 3x32x32)
- Saving the full model instance instead of the state dict

- Using a custom model class not in the allowed list
- Mismatched model-name field

Scoring

All submissions are automatically validated and evaluated on the hidden test set.

- **Public leaderboard:** based on 30% of models
- **Private leaderboard:** final ranking based on the remaining 70%

This means that **optimizing only the leaderboard score does not automatically give you a good final result**. Your results must generalize for all models. The leaderboard score reflects the **score** achieved by the submission. Higher is better.

Leaderboard

Results are published at: http://34.63.153.158/leaderboard_page

The leaderboard shows the best result per team only. If your current submission score is lower than the score saved in the leaderboard, the score will not be updated.

Points to note regarding submission:

You can only submit a CSV with scores **every 60 minutes**. If your submission is malformed or wrong, you should get a comprehensive error message. However, even if you fail, you get a **2 minute** cooldown.

Read the following instructions carefully:

Additionally, you are required to submit a **ZIP file** to CMS that contains your code and a report that explains your approach to the task. This file should contain:

1. Report

- Matriculation number and **CMS team Id** in the report
- An Introduction:
 - A short paragraph explaining **your understanding of the task** written by your own words.
- A Main body:
 - Explaining **your approach to the task** including the details about the hyperparameters and structural choices you have made, as well as the reasons you made that choice.
 - Explaining **key results** that you have obtained on the public leaderboard.
- A Conclusion
 - In your conclusion try to be goal oriented. In other words, try to answer this question: now that we have trained a robust model, what practical implications does adversarial vulnerability have in real-world systems and why should we be concerned?

2. Code Repository

- Containing all your code files, well documented and clean
- Every experiment mentioned in the report must have corresponding code in the repository
- **Avoid including large model artifacts in your repository.**

- Add a README.md that ONLY explains how to **recreate** your best leaderboard result
- You are only supposed to make one submission per group

The report should NOT be more than **2 pages** long. The submitted zip file that does not contain the code and/or the report will NOT be considered for evaluation. For the report you SHOULD use the [template for ICLR](#) on overleaf. Note that there is no need for an abstract and you should skip it.

The submission to the leaderboard will end automatically on **16.06.2026 23:59** and after that you will not be able to submit any results to the leaderboard. The same date also applies to CMS. As a result, make sure to submit your results and report on time. Thereafter, you will be able to check your evaluation results performed on the held-out set on the leaderboard. No late submissions will be accepted via email (no exceptions).

References

- A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards Deep Learning Models Resistant to Adversarial Attacks,” in International Conference on Learning Representations (ICLR), 2018. Available: <https://openreview.net/forum?id=rJzlBfZAb> (PGD attack)
- I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and Harnessing Adversarial Examples,” arXiv preprint arXiv:1412.6572, 2015. Available: <https://arxiv.org/abs/1412.6572> (FGSM attack)
- A. Athalye, N. Carlini, and D. Wagner, “Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples,” in Proceedings of the 35th International Conference on Machine Learning (ICML), vol. 80, pp. 274–283, 2018. Available: <https://proceedings.mlr.press/v80/athalye18a.html>